

Deep Learning Project

McKay Shields

December 2024

1 Introduction

As a part of the Field Robotic Systems (FRoSt) Labs at BYU, I have been interested in working with underwater robotics. Though this is not my area of work, one of the focuses of the lab is to explore shipwrecks on the ocean floor. This can be used for exploration or rescue purposes. I decided to create a neural network that could detect sunken ships from sonar scans of the sea floor. The data was collected over the last few years at the Thunder Bay National Marine Sanctuary in Alpena, Michigan. Those at the University of Michigan then helped label the data, pixel by pixel. Each image is a frame of the sonar scan. Although it almost looks like grayscale, the images have turned sonar data into RGB values, meaning these are colored images. The labels are binary tensors where a 0 indicates no ships and a 1 means there is a ship. This is represented as a black background with white in the corresponding locations of ships. With clear labels for the images, this will be a supervised learning classification problem.

The dataset was partitioned into training and test sets. The data was split so that 80% of the images were used for training and 20% for testing. This partition was already created outside the notebook. Then the training data was split 80-20 into training and validation sets. This split ensures that the model has sufficient examples to learn from during training while retaining unseen data to evaluate performance and generalization. Data augmentation techniques such as random flips, rotations, and rescaling could also be applied during training to improve robustness.

2 The Model

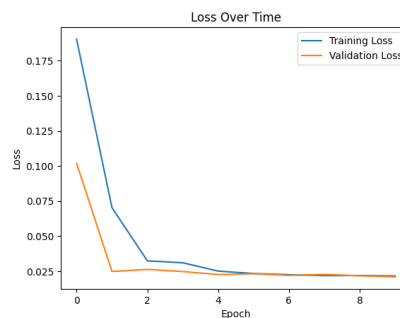
The model I created is a custom-built convolutional neural network. Each layer uses padding to maintain spatial dimensions. ReLU activations introduce non-linearity and a bilinear up-sampling operation ensures that the output segmentation map is resized to match the input di-

mensions (128×128). This simple architecture balances computational efficiency (when runtime is a concern) and classification accuracy. After some trial and error, I learned that although things like drop outs or batch norms helped in validation, the most consistent results used a much simpler network.

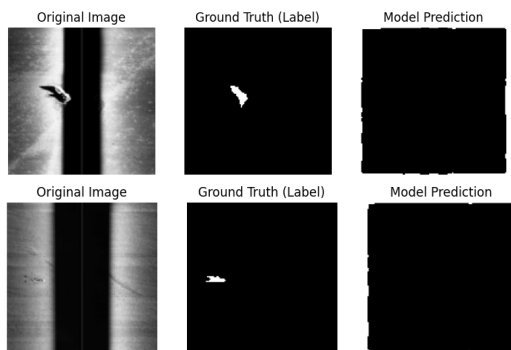
For training, Binary Cross-Entropy Loss was used. An Adam optimizer, with a learning rate of 0.001, was selected for training for its efficient convergence properties. During inference, the output logits are passed through a sigmoid activation function to generate probabilities. A threshold is applied to classify each pixel as 0 or 1. In this approach, no pre-trained weights were used. The model was trained from scratch on the dataset. Incorporating pre-trained weights in future iterations could potentially improve performance.

3 The Results

After several parameter changes, it's clear there were signs of overfitting. This was seen from the training and validation loss curves. During training, the model's training loss decreased steadily. However, when the classifier ran on the test data, it classified almost every pixel, there were some cases where it was close in the classification. The model consistently classified most pixels as "background" (0's) during validation, likely because the rare positive (1) pixels were not generalized well. Regularization techniques like dropout helped but were not fully sufficient.

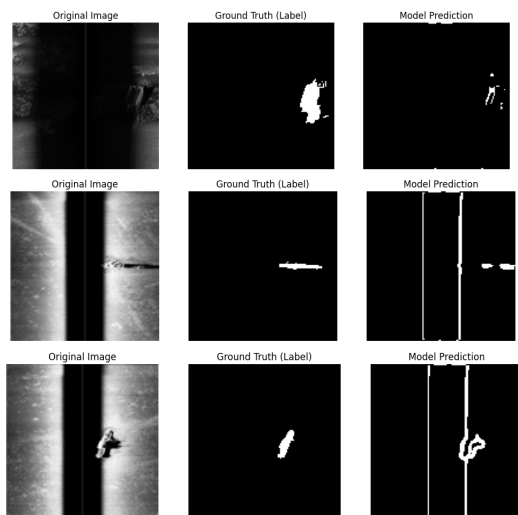


3.1 Examples of Empty Results



After the CNN was not yielding satisfactory results. I tried building and implementing a UNet, using similar architecture to the cancer identification lab from this class. The two problems are similar and I thought this would work better. It did seem to have fewer false negatives but often calculated the borders of the sonar scans as ships. I didn't understand this architecture as well so I abandoned it and returned to fine-tuning the results of the CNN.

3.2 UNet Results

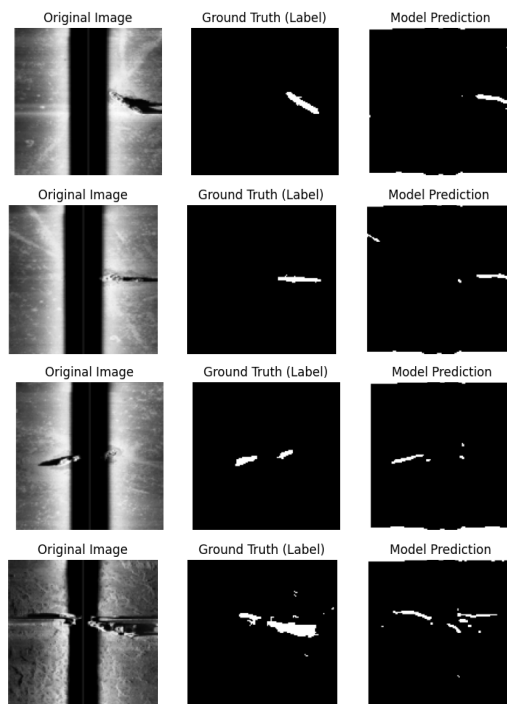


After spending a significant amount of time adjusting paddings, kernel sizes, and learning rates to improve my model, I was still struggling to get meaningful results. The outputs were consistently unsatisfactory, often classifying all pixels as 0, with almost no positive predictions. I thought I was missing something in the network architecture itself. After some analyzing the outputs of the model, I decided to create a histogram of all the probabilities my model was outputting.

All of these probabilities are less than 0.5, though you can see a small cluster of pixels with

probability in the 0.03 range. When I took a closer look at the logits values and their distribution, it dawned on me that most of the logits were negative, making all the probabilities less than 0.5. I realized I had arbitrarily set the threshold, assuming it was the default choice for binary classification. I lowered this value to around 0.02. This yielded far more correct classifications.

3.3 Some Promising Results



4 Conclusion

The results above are decently good at classifying where sunken ships are (even if the exact size and shape of the boat is not accurate). However, these screenshots have been handpicked as some of the better examples. And despite this, these results are not perfect. There are patches falsely classified as parts of a ship and patches that are completely missed. A few from the test data set classified the border between the left and right scans as a ship (as seen in the UNet Results). Significant progress has been made on solving this problem. These preliminary results show that neural networks are indeed the right direction in classifying underwater sonar scans. Future work could be done to make a deeper model that can find the nuances features of this ship classification.

5 Time Log

Date	Hours Spent	Activity
11/30	2.0	Read about data gathered
12/2	1.5	Downloaded and prepared data
12/3	2.0	Built notebook, uploaded dataset, created dataloaders
12/5	3.5	Started building network
12/7	1.5	Built network
12/10	2.0	Finished first attempt and trained
12/12	4.0	Trained classifier network, experimented with different layer types
12/13	5.0	Read about, built, and tested U-Net
12/14	1.5	Fine tuned layers and parameters for classifier
12/16	6.5	Adjusted sigmoid parameters, took screenshots of results, wrote project report
Total:	30.5 hours	